

YipitData | Data Engineering Assignment

Objective

Your objective is to build a local Python data pipeline that turns messy technology-news articles into two outputs: a queryable company ARR observations over time, and a vector search index to support semantic search.

At YipitData, we work with many kinds of messy source data and turn them into datasets that investors can use. This exercise uses a small synthetic news dataset to mirror that work: parse inconsistent source fields, join company context, preserve lineage, and produce outputs that support analysis and semantic search.

Problem Statement

The source data is a synthetic technology-news dataset:

- Each article may contain a messy **revenue** value.
- For this assignment, assume every valid **revenue** value represents a **reported ARR observation** for the article's company as of the article's **published_date**. The source column is still named **revenue**; your cleaned/modeled outputs should make the ARR interpretation clear.

Build a pipeline and data model that preserve source lineage and make the cleaned observations usable for analysis. The dataset is small enough to run locally, but design it as if new article batches would continue arriving.

Example:

If an article for Snowflake published on **2023-05-12** contains **€2.1B**, treat it as an ARR observation for Snowflake as of **2023-05-12**, convert it to USD, and keep the original string available for audit.

Input Datasets

You will receive two files:

- **tech_news.csv** - 750 technology-news article records
- **company_metadata.json** - company metadata for enrichment

The data is intentionally imperfect. Clean what you can, retain source lineage, and document assumptions.

Constraints / Assumptions

- All valid **revenue** values represent reported ARR observations.
- Source data is synthetic and intentionally messy.
- The pipeline should run locally.

Your Task

Build one local Python pipeline with the capabilities below. Keep the implementation small enough to run locally. Choose the table structure, file layout, and naming that make the model easiest to query and audit.

1. Clean and Enrich the Source Data

Create reusable cleaning and enrichment functions for the article and company metadata files.

Revenue / ARR Cleaning

Clean the source **revenue** column and produce a normalized USD value.

Requirements:

- Convert all amounts to USD.
- Handle EUR, GBP, and JPY.
- Handle ranges such as **\$10M - \$20M** by taking the midpoint.
- Parse formats such as **5.2B**, **\$5,200,000,000**, **5.2 billion**, and **500M USD**.
- Handle missing, null, **N/A**, and **Not disclosed** values.
- Return parsed values as integers.
- Do not silently treat missing or undisclosed values as valid ARR observations in the modeled ARR output.

Currency conversion rates for this exercise:

- EUR to USD: multiply by **1.1**
- GBP to USD: multiply by **1.27**
- JPY to USD: divide by **150**

Date Normalization

Clean the **published_date** column.

Requirements:

- Handle multiple date formats, including ISO, US, and EU-style formats.
- Return a datetime-compatible value.

- Extract year, quarter, and month for filtering and analysis.
- Handle invalid dates and missing values.
- Document how you handle ambiguous numeric dates.

Category Standardization

Normalize the `category` column.

Requirements:

- Map similar categories into a consistent taxonomy.
- For example, `AI/ML`, `Artificial Intelligence`, and `Machine Learning` should map to a common value such as `AI_ML`.
- Provide the mapping in your code.

Company Metadata Integration and Validation

Validate and enrich article company names with the metadata file.

Company name validation:

- Validate that article company names exist in the metadata.
- Log, flag, or export companies that cannot be matched.
- Consider fuzzy or alias matching for slight variations, such as `Amazon Web Services` vs `AWS`.

Metadata enrichment:

- Join company metadata to articles by company name or resolved company identity.
- Include metadata fields in enriched outputs, including industry, founded year, headquarters, employee count, public/private status, and stock ticker when available.
- Add `company_age`, calculated from `founded_year` and the article `published_date`.
- Add `company_size_category`.

Company size thresholds:

- `Small`: fewer than `10,000` employees
- `Medium`: `10,000` through `30,000` employees
- `Large`: more than `30,000` employees

Industry-based filtering:

- Use the metadata `industry` field as an additional filter option.

- The metadata includes industries such as **AI/ML**, **Cloud Computing**, and other technology sectors.
- Consider both article **category** and company **industry** when filtering AI-related records.

2. Model Company ARR Observations

Design a queryable model that supports company ARR observations over time. Materialize the model as warehouse-style tables and export each table as its own CSV file. The model should make clear the grain of each table, the identifiers and relationships you chose, how ARR observations link back to source articles, how invalid or missing ARR records are handled, and how someone should derive latest ARR or quarterly ARR views.

You may use bronze/silver/gold naming if helpful. Use the structure you think is clearest.

The model should support these query patterns:

- **View ARR observations for a company over time - this is the most important use case.**
- Find the source article for an ARR observation.
- Filter articles by date, category, industry, and ARR threshold.
- Re-run the pipeline without creating duplicate modeled records.
-

3. Export the AI Article Dataset and Modeled Outputs

Export a CSV named *ai_articles_enriched.csv*.

It should include all articles that match:

- article category indicates AI / Machine Learning, OR company industry indicates AI / ML
- published between 2022 and 2024
- valid ARR greater than **\$50M** USD

Include these columns:

- **article_id**
- **title**
- **company_name**
- **published_date**
- **category**
- **arr_usd**
- **summary**
- **url**
- **industry**

- `founded_year`
- `headquarters`
- `employee_count`
- `is_public`
- `stock_ticker`
- `company_age`
- `company_size_category`
- `embedding`

Export each modeled data warehouse table as its own CSV file (e.g., a separate file for your company dimension). A local DuckDB database is optional, but the per-table CSV exports are required. The exported tables should include enough modeled article, company, and ARR-observation data to inspect lineage, debug failed parses, and query ARR over time.

The AI article CSV is a required downstream export. It does not replace the modeled table CSVs.

[BONUS] Add Semantic Search and Hybrid Querying

Implement semantic search over articles using embeddings.

Text Embedding Generation

Requirements:

- Generate embeddings for article `title + summary`.
- Use `sentence-transformers` with `all-MiniLM-L6-v2`, or a reasonable equivalent.
- Store embeddings in a reusable form, such as numpy arrays plus a serialized output.

Vector Similarity Search

Requirements:

- Implement cosine similarity search.
- Provide a function such as `find_similar_articles(query_text, top_k=5)`.
- Return article IDs and similarity scores.

DuckDB Integration and Hybrid Search

Requirements:

- Load cleaned/enriched data into DuckDB, unless you have a clear reason to choose another local analytical store.
- Store embeddings using DuckDB's ARRAY-like representation or JSON.
- Provide a function for hybrid search: SQL filters plus vector similarity.

- Example: find AI-related articles from 2022-2024 with ARR greater than \$50M, similar to a query or article.
- A persisted DuckDB database file is optional; the modeled table CSV exports in the next section are required.

Add `top_similar_articles` as the IDs of the top 3 most similar articles, excluding the current article. For each article record, you should find the top 3 other articles that have the most semantic similarity (using similarity search), and include those in the `top_similar_articles` field.

Expected Deliverables

Submit:

- source code, including test coverage
- `README.md`
- Data Architecture Document
- `requirements.txt`
- generated CSV outputs: *ai_articles_enriched.csv* and one CSV file for each modeled data warehouse table
- optional local DuckDB database file

The README should explain:

- installation
- how to run the pipeline
- how to regenerate all required CSV outputs
- example queries or usage
- system requirements

The Data Architecture Documentation should explain the data model you chose and how this pipeline would run reliably beyond a local batch.

Evaluation Criteria

We will evaluate:

- quality of the data model and architecture
- correctness and reliability of the pipeline
- handling of messy and invalid records
- clarity of lineage from analytical outputs back to source records
- usefulness of the semantic search implementation
- handling of backfills and schema changes
- clarity of communication

Do not treat article ARR values as company master data without source lineage and caveats.

Live Review Session (45 minutes live review + coding)

- Be prepared to answer questions from the interviewer about your architecture and design decisions
- Be prepared to code follow-up features live based on interviewer questions
- Discuss any additional assumptions, trade-offs, or considerations you made in the process

Submission

Please submit any deliverables ahead of the scheduled live session. **You are highly recommended to use AI coding agents to assist with the exercise and produce the highest quality version you can.** You will still be expected to know all the implementation details, answer questions about the architecture / design decisions live, and code follow-up features in the live session.